부하 테스트 결과 보고: 잠금 없는 읽기 개선 전후 비교

Oct 19, 2026 · @원우님

한눈에 보기

화면 표시용 읽기와 방 목록의 게임 확인에서 게임 잠금을 뺐더니, 방 10개 부하에서 Tomcat 스레드 포화(200/200)가 사라지고 로비 방 목록 p99가 13.4초에서 215ms로 약 62배 빨라졌다.

- **문제:** 게임 하나의 잠금 경합이 스레드를 붙잡아, 잠금과 무관한 로그인·방 목록까지 느려졌다. 방 10개에서 스레드 200개가 모두 차고 로그인 p99가 1.18초가 됐다.
- **개선:** 상태 저장소가 Redis라 매번 새 스냅샷을 돌려줄 때는 읽기에 잠금을 잡지 않는다(브랜치 `perf/snapshot-read`). 쓰기는 그대로 잠금 안에서 처리한다.
- **결과:** 방 10개에서 바쁜 스레드 200 → 74, 로그인 p99 1.18s → 92ms, 드롭 요청 239 → 0. 오류는 개선 전후 모두 0건이다.
- **남은 과제:** 잠금을 계속 쓰는 투표 p99가 862ms로, 5방(약 370ms)보다 높다. 다음 병목은 쓰기 경합이다.

무엇을 바꿨나

Redis 저장소는 게임 상태를 키 하나(`game:v1:state:{id}`)에 JSON으로 두고, 읽을 때마다 새 객체로 해석해 돌려준다. GET 한 번은 원자적이라 잠금 없이 읽어도 반쯤만 바뀐 상태를 볼 일이 없다. 이 성질을 이용해 읽기를 잠금 밖으로 누었다.

변경	내용	이유
`GameRepository.returnsSnapshots()`	기본값 false, Redis 저장소만 true. `ChangeEventGameRepository`는 감싼 저장소의 값을 그대로 전달	인메모리 저장소는 살아 있는 객체를 돌려주므로, 잠금 없이 읽으면 쓰는 도중의 값을 볼 수 있다

변경	내용	이유
`SnapshotRead.of(...)` (신규)	스냅샷 저장소면 그냥 읽고, 아니면 기존처럼 잠금 안에서 읽는다	single 모드(인메모리) 동작을 바꾸지 않기 위해
표시용 읽기 5곳	`GameService.getState`, `getMyRole`, `getResult`, `NightService.getNightResult`, `VoteService.getExecutionResult`가 `SnapshotRead` 사용	화면에 보여주기만 하는 읽기는 쓰기와 순서를 맞출 필요가 없다
`keepsRoomInGame` (방 목록)	먼저 잠금 없이 확인해서 게임이 살아 있으면 바로 true. 게임이 없거나 끝났을 때만 잠금 안에서 다시 확인	방 목록이 게임 잠금을 하나씩 차례로 잡는 것이 로비 지연의 원인이었다. 방을 고아로 정리하는 드문 경우만 정확성을 위해 잠금을 유지
`SnapshotReadTest` (신규, 7개)	스냅샷 저장소는 잠금 0회, 인메모리는 잠금 1회, 방 목록 분기별 잠금 횟수 검증	동작 보장과 회귀 방지

일부러 바꾸지 않은 것: 채팅 전송의 규칙 확인(죽은 사람·밤 채팅 제한)은 잠금을 유지했다. `ChatGameLockTest`가 이 규칙을 잠금 안에서 확인하도록 정해 두었고, 바꾸려면 팀 합의가 필요하다. 투표·밤 능력 같은 쓰기도 그대로다.

측정 방법

같은 노트북에서 같은 조건으로 dev(개선 전)와 `perf/snapshot-read`(개선 후)를 번갈아 재고, 잠금 보유 시간이 같은 회차끼리만 비교했다.

시나리오 (`loadtest/lock-contention.js` , k6, 2분): 방마다 8명이 VOTE 단계에 있는 상태에서 다음 부하를 동시에 건다.

시나리오	부하	잠금 사용 (개선 전 → 후)
투표 몰림	2초마다 방당 8명 동시 투표	잠금 → 잠금
채팅 폭주	1인당 초당 1건	잠금 → 잠금
상태 재조회	1인당 초당 1건	잠금 → **없음**
카나리아	초당 5건, 게스트 로그인과 방 목록 번갈아	로그인은 없음, 방 목록은 게임마다 잠금 → **거의 없음**

카나리아는 대조군이다. 로그인은 게임 잠금을 안 쓰므로, 로그인이 느려지면 잠금이 아니라 스레드 부족이 원인이다.

환경: Spring Boot 서버 `SERVER_MODE=multi` (게임 상태·잠금·타이머 모두 Redis), 프로필 `local,loadtest` , Tomcat 스레드 200개. 게임 Redis는 Toxiproxy(지연 없음)를 거쳐 연결하고, Prometheus 1초 수집과 Grafana로 서버 지표를 봤다. k6, 서버, Redis가 모두 한 대의 Windows 노트북에서 돈다.

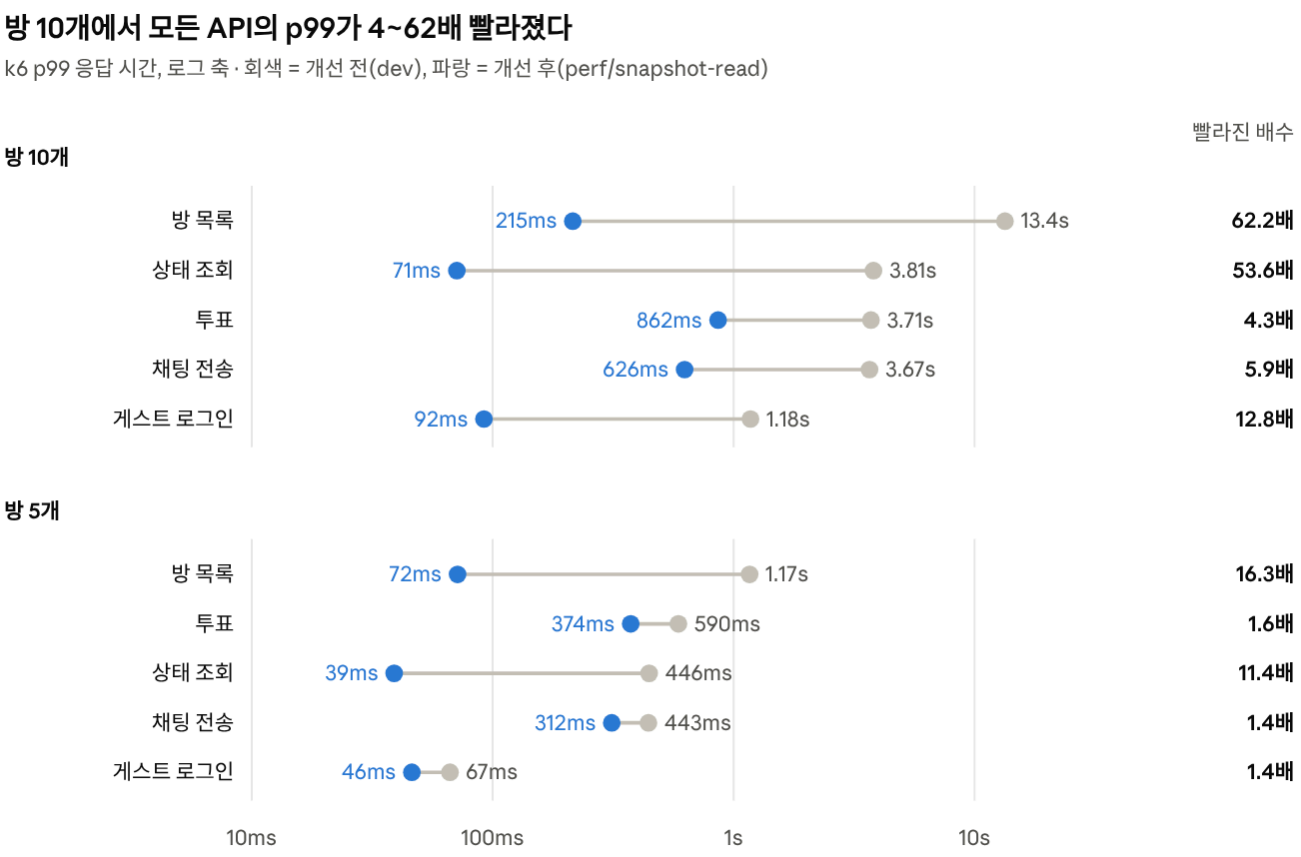
회차 통제:

1. 매 회차 서버 종료 → `FLUSHALL` → 서버 재시작 (브랜치 전환 후에는 재빌드)
2. 전원 연결, 고성능 모드, IntelliJ 인덱싱·브라우저 등 다른 부하 없음
3. 조건마다 2회 측정, 회차 사이 쿨다운

이상치 판단 기준: 잠금 보유 p99. 보유 시간은 잠금 안에서 Redis를 읽고 쓰는 실제 작업 시간이고, 이번 개선은 이 작업을 바꾸지 않았다. 그래서 보유 p99가 평소(약 90~170ms)의 2배 가까이 뛴 회차는 코드가 아니라 노트북 상태 문제로 보고 제외했다. 제외한 회차는 2건이고, 둘 다 보유 p99가 340ms 안팎이었다.

결과

방 10개에서 모든 API가 빨라졌고, 잠금을 뺀 방 목록(62배)과 상태 조회(54배)가 가장 크게 줄었다. 방 5개에서도 방향은 같지만 폭은 작다.



k6 결과 요약 · 개선 전은 5방·10방 각 2회 평균, 개선 후는 5방 2회 평균·10방 정상 1회

서버 쪽 지표도 같은 방향이다. 범위는 2회 측정의 최솟값~최댓값이다.

지표 (서버)	5방 개선 전	5방 개선 후	10방 개선 전	10방 개선 후
바쁜 Tomcat 스레드 최대 (/200)	31~33	10~19	**200 (포화)**	**74**
잠금 대기 p99	597~641ms	495~510ms	3,496~3,554ms	857ms
잠금 보유 p99	87~131ms	105~143ms	160~172ms	172ms
k6 드롭 요청	0	0	212~266	0
503 · 기타 오류	0	0	0	0

잠금 보유 p99가 개선 전후 같은 범위에 있으므로, 잠금 안의 작업 속도는 같고 줄 서는 요청 수만 줄었다는 뜻이다. 10방 개선 전에 503이 0건인 것은 정상이어서가 아니다. 요청이 잠금 제한 시간(10초)에 닿기 전에 Tomcat 대기열에서 먼저 줄을 섰기 때문이다.

해석: 왜 이렇게 달라졌나

잠금을 기다리는 요청 수를 줄이자 게임별 잠금 줄이 짧아졌고, 줄에서 스레드를 붙잡고 있던 요청이 사라져 서버 전체가 풀렸다.

1. **게임마다 잠금 줄이 하나다.** 한 게임의 잠금은 한 번에 한 요청만 처리한다. 초당 요청 수 × 평균 보유 시간이 1에 가까워지면 대기가 급격히 길어지고, 1을 넘으면 끝없이 쌓인다. 개선 전에는 게임당 초당 약 20건(채팅 8 + 상태 8 + 투표 4)이 줄을 섰고, 개선 후에는 약 12건(채팅 8 + 투표 4)만 선다.
2. **줄에서 기다리는 동안 스레드를 붙잡는다.** Redisson `tryLock`은 최대 10초까지 요청 스레드를 멈춰 두고 기다린다. 개선 전 10방에서는 대기 p99가 3.5초라 스레드 200개가 모두 찼고, 잠금과 무관한 로그인도 Tomcat 대기열에서 순서를 기다렸다(1.18초).
3. **방 목록은 줄을 여러 번 섰다.** 방 목록 한 번에 진행 중인 게임 10개의 잠금을 차례로 잡아, 대기 시간이 게임 수만큼 더해졌다. 그래서 10방에서 13초가 걸렸고, 개선 후에는 줄을 서지 않아 62배 빨라졌다.
4. **코드를 안 바꾼 채팅·투표도 빨라졌다.** 같은 줄에 서던 상태 조회가 빠지면서 앞사람이 줄었기 때문이다(10방에서 4~6배).
5. **효과는 부하가 클수록 커진다.** 방 목록 개선 폭은 5방 16배, 10방 62배다. 포화 구간에서는 줄을 조금만 줄여도 대기 시간이 크게 줄기 때문이다.

그러나 10방은 여전히 임계점 근처다. 제외한 개선 후 10방 1회차는 보유 p99가 342ms로 평소의 2배였고, 이때는 개선된 코드도 스레드 200/200, 로그인 p99 7.3초, 503 6건으로 무너졌다. 쓰기 작업이 조금만 느려져도 다시 붕괴할 수 있다는 뜻이다.

측정의 한계와 주의점

결론의 방향은 믿을 만하지만, 배수 같은 정확한 숫자는 이 노트북과 이 시나리오에서만 성립한다.

- **개선 후 10방의 정상 회차는 1회뿐이다.** 3회차로 받은 출력은 요청 수(25,039건)까지 2회차와 완전히 같아 같은 출력으로 보고 넣지 않았다. 재현성을 보려면 1회 더 필요하다.
- **부하 발생기와 서버가 같은 노트북에서 돈다.** 서버가 느려지면 k6가 가상 사용자를 더 띄워(제외 회차 1,220개) CPU를 더 쓰고, 그러면 서버가 더 느려지는 악순환이 생긴다. 붕괴 회차가 실제 운영보다 나쁘게 나왔을 수 있다.
- **노트북 상태에 민감하다.** 같은 코드에서도 보유 p99가 87~342ms로 흔들렸다. 연속 측정에 따른 발열과 스로틀링이 주된 원인으로 보인다. 보유 p99를 측정 환경 건강도 지표로 쓴 것은 그래서다.
- **Redis 지연이 0에 가깝다.** 로컬 Redis라 왕복 시간이 짧다. 실제 클라우드 Redis는 왕복이 더 길어 잠금 보유 시간이 늘고, 임계점이 더 적은 방 수에서 온다.
- **클라이언트 패턴을 단순화했다.** 상태 조회 초당 1건, 채팅 초당 1건은 스트레스용 가정이다. 실제 Unity 클라이언트는 WebSocket을 주로 쓰고 끊겼을 때만 폴링한다.
- **서버 요청 타이머는 Tomcat 대기열 시간을 빼고 잔다.** 사용자가 느끼는 지연은 k6 값으로 읽었다.

추가 보완할 점

남은 병목은 쓰기 경합이다. 다음 개선은 잠금을 잡는 요청 수나 보유 시간을 줄이고, 붕괴 시에도 다른 기능을 지키는 쪽이다.

우선	보완할 점	기대 효과	비고
1	**채팅 규칙 확인을 스냅샷으로** (발신자 생존·단계 확인을 잠금 밖에서)	게임당 잠금 요청 초당 12건 → 4건. 투표 대기가 크게 줄 것	`ChatGameLockTest` 규칙 변경이라 팀 합의 필요. 단계가 바뀌는 순간 채팅 1건이 통과할 수 있는 수준의 오차 허용 여부
2	**잠금 대기를 짧게, 빨리 실패** (`wait-timeout` 10초 → 1~2초, 초과 시 503)	붕괴 회차처럼 스레드가 잠금 대기에 묶이는 것을 막아 로그인·로비를 지킴	클라이언트가 503을 받으면 재시도하도록 맞춰야 함
3	**게임 요청 전용 스레드 상한 (bulkhead)**	게임 API가 스레드를 다 쓰지 못하게 해 로그인·방 목록 보장	2와 함께 쓰면 효과가 크다
4	**잠금 안의 Redis 왕복 줄이기** (읽기·쓰기를 하나의 스크립트나 파이프라인으로)	보유 시간 감소 → 임계점에서 멀어짐	보유 시간이 어디서 쓰이는지 먼저 측정
5	**투표 몰림 처리** (투표를 받아두고 마감 때 한 번에 집계)	동시 투표 8건이 줄서는 대기(투표 p99 862ms) 제거	구조 변경이 커서 장기 과제

측정 보완:

- [ ] 개선 후 10방을 쿨다운 후 1회 더 측정해 재현성 확인
- [ ] 방 수를 15·20개로 늘려 개선 후의 새 한계점 찾기
- [ ] Toxiproxy로 Redis 지연(예: 2ms, 5ms)을 넣어 클라우드 환경에 가까운 조건에서 재측정
- [ ] k6를 다른 장비에서 돌려 부하 발생기와 서버의 CPU 경합 분리
- [ ] 인메모리 잠금(`LocalGameLock`)에도 같은 지표를 달아 single 모드와 비교
- [ ] `.\gradlew.bat test`로 `SnapshotReadTest` 포함 전체 테스트 통과 확인 후 PR 제출

전체 회차별 원자료는 프로젝트 문서 `claude/부하테스트_측정기록_lock-contention.md`에 있다.